

CL-TAV algorithm specification package

CL-TAV specification (CR-2026-014 / DD-036)

This package typesets the CL-TAV algorithm specification. The main text gives the top-level session and the three core mechanisms (test selection, timed observation interpretation, history-compatible update). The appendix gives the S2/S6/S7–S8 coordination wrappers, the typed records, the responsibility map and the unresolved-kernel register. It is not a second program and selects no solver technology. Step labels S0–S10 are the stable bilingual mapping; typeset line numbers are not cited.

Algorithm 1: CL-TAV top-level session (ALG-CLTAV-01)

```
Require:  $H_0 = \{h_{\text{normal}}\} \cup H_{\text{single}}$ ; SessionContext  $\Gamma$  (unique state); HistoryHandle  $\eta$ ; exactly one resource mode  
BUDGET( $c_{\min}, B$ ) xor ROUNDS( $K_{\max}$ ); action library  $L$ ; uncertainty reference  $U$   
Ensure: stop class, final  $H$ ,  $\eta$ , session trace; singleton  $h_{\text{normal}}$  is not protocol PASS  
// S0 INITIALIZE: state, candidates, history, resource, retry, uncertainty reference  
1  $(\Gamma, \eta, H) \leftarrow \text{Initialize}(H_0, L, U, \text{resourcePolicy})$ ;  
2 while true do  
  // S1 ENTRYSTOP: maps to the existing A2--A5 / P1--P5 decision table (Appendix C)  
3  $\text{stop} \leftarrow \text{IF-RESOURCE-STOP}(\Gamma, H, \eta)$  with IF-EQUIV( $\Gamma, H, \eta$ );  
4 if  $\text{stop} \neq \text{undecided}$  then return  $\text{Finish}(\text{stop}, \Gamma, \eta, H)$ ;  
  // S2 PREDICTCURRENT: typed prediction; the gap tag is handled before any projection  
5  $\text{pred} \leftarrow \text{PredictCurrent}(\Gamma, \eta, H, U, L)$ ;  
6 if  $\text{pred.status} = \text{GAP}$  and not  $(\text{RecoverEligible}(\Gamma, L)$  or  $\text{PrepEligible}(\Gamma, L))$  then return  $\text{Finish}(\text{pred.reason}, \Gamma, \eta, H)$ ;  
  // S3 SELECTANDADMIT: tagged exit or spec error returns with its reason  
7  $\text{dec} \leftarrow \text{SelectAndAdmit}(\Gamma, \eta, H, L, \text{pred})$ ;  
8 if  $\text{dec.kind} \in \{\text{EXIT}, \text{SPEC-ERROR}\}$  then return  $\text{Finish}(\text{dec.reason}, \Gamma, \eta, H)$ ;  
  // S3-SNAP SELECTSNAPSHOT freezes  $\xi$  (actionId, actionKind, qUsedAtSelect, historyVersion,  
  classesUsedAtSelect, uncertaintyRef); the action kind is  $\text{dec.actionKind}$ , never the exit/action  
  discriminant  
9  $\xi \leftarrow \text{SelectSnapshot}(\text{dec.actionId}, \text{dec.actionKind}, \Gamma.\text{currentSummary}, \eta.\text{version}, \text{dec.classesUsed}, U)$ ;  
  // S4 CHARGEONCE: one charge under the selected mode; retry cap  $\neq$  budget exhaustion  
10  $\Gamma \leftarrow \text{ChargeOnce}(\Gamma, \xi)$ ;  
  // S5 EXECUTEANDRECORD: execute the snapshot action exactly once  
11  $\text{er} \leftarrow \text{IF-EXECUTE-RECORD}(\Gamma, \xi)$ ;  
  // S6 INTERPRETOUTCOME: ALG-CLTAV-03 coordinates; ALG-CLTAV-06 interprets timing  
12  $z \leftarrow \text{InterpretOutcome}(\text{er}, \xi, \Gamma, \eta, U)$ ;  
  // S7--S8 RESOLVEOUTCOME: a total Resolution; only COMMIT reaches S9  
13  $\delta \leftarrow \text{ResolveOutcome}(z, \xi, \Gamma, \eta)$ ;  
14 if  $\delta.\text{control} = \text{STOP}$  then return  $\text{Finish}(\delta.\text{reason}, \delta.\Gamma, \delta.\eta, H)$ ;  
15 if  $\delta.\text{control} = \text{RETRY}$  then continue;  
  // adopt the returned contexts, then commit  
16  $\Gamma \leftarrow \delta.\Gamma$ ;  $\eta \leftarrow \delta.\eta$ ;  
  // S9 the S9 guard is an executable condition, not a comment: a valid observation or a confirmed successor  
  summary  
17 if  $z.I_z$  is a valid observation class or  $z.\text{summaryConfirmed}$  is true then  
18  $(\eta, H, \Gamma) \leftarrow \text{CommitCompatibleUpdate}(\xi, \delta.\text{outcome}, \Gamma, \eta, H)$ ;  
  // S10 POSTUPDATESTOP: existing post-update stop priority; otherwise the next iteration is S1  
19  $\text{stop} \leftarrow \text{IF-RESOURCE-STOP}(\Gamma, H, \eta)$  with IF-EQUIV( $\Gamma, H, \eta$ );  
20 if  $\text{stop} \neq \text{undecided}$  then return  $\text{Finish}(\text{stop}, \Gamma, \eta, H)$ ;
```

Algorithm 2: SelectAndAdmit: test selection and resource admission (ALG-CLTAV-05, S3)

Require: Γ ; η ; current H ; action library L ; typed $pred$ (PredictionResult)
Ensure: a Decision: kind ACTION / EXIT / SPEC-ERROR with actionKind, actionId, classesUsed and reason
// a GAP prediction carries no classesByTest: handle the tag first, and let eligible Prep/Recover fall back

```
1 if  $pred.status = GAP$  then
2   if PrepEligible( $\Gamma, L$ ) then return Decision(kind = ACTION, actionKind = PREP, the eligible Prep,  $\cdot, \cdot$ );
3   ;
4   if  $\Gamma.qStatus = UNKNOWN$  and RecoverEligible( $\Gamma, L$ ) then return
      Decision(kind = ACTION, actionKind = RECOVER, the eligible Recover,  $\cdot, \cdot$ );
5   ;
6   return Decision(kind = SPEC-ERROR, reason =  $pred.reason$ ); the gap is a spec error only when no fallback exists;
// OK prediction: read classesByTest only now; UNKNOWN recovery eligibility never reads the untrusted old  $q$ 
7  $A \leftarrow \{t \in L : \text{Affordable}(t, \Gamma)\} \cap (\Gamma.qStatus = UNKNOWN ? \{\text{eligible Recover}\} : \{t : \text{Enabled}(t, \Gamma)\})$ ;
8  $C_t \leftarrow \{H \cap C : C \in pred.classesByTest[t], H \cap C \neq \emptyset\}$  for each TEST  $t \in A$ ;
9 if a TEST  $t \in A$  has declared classes but  $C_t = \emptyset$  then return
   Decision(kind = SPEC-ERROR, reason = per-test prediction gap for  $t$ );
10 ;
// IF-SELECT-ADMIT is the single selection authority: exclusive kind; one-step minimax  $s(t) = \max_{C \in C_t} |C|$  over
   currently valid nonempty classes; cost/id tie-break; never argmin  $\emptyset$ ; Prep/Recover are not observation
   minimax
11  $(k, S, t^*, admit) \leftarrow \text{IF-SELECT-ADMIT}(A, \Gamma.currentSummary, \{C_t\}, \Gamma, \eta, H)$ ;
12 if  $k = ADMIT$  then
13   return Decision(kind = EXIT, reason = the existing A2–A5 reason);
14 else if  $k = PREP$  then
15   return Decision(kind = ACTION, actionKind = PREP, actionId =  $t^*$ , classesUsed =  $S$ );
16 else if  $k = RECOVER$  then
17   return Decision(kind = ACTION, actionKind = RECOVER, actionId =  $t^*$ ,  $\cdot$ );
18 return Decision(kind = ACTION, actionKind = TEST, actionId =  $t^*$ , classesUsed =  $C_{t^*}$ ); consume the returned  $t^*$ , do not
   re-select;
```

Algorithm 3: InterpretTimedObservation: timed observation kernel (ALG-CLTAV-06, S6 kernel)

Require: the single ownership result and the raw measurement interval from IF-OBS-INTERPRET; SelectSnapshot ξ ; named obligation inputs N_r (allowed interval) and D_r (physical domain); declared match policy $Match_r$; declared uncertainty ε
Ensure: a TimingResult: verdict in {PASS, FAIL, INCONCLUSIVE, ERROR}; the single pairing evidence; the interval used; validity
// resolve N_r , D_r and ε from the named obligation and instance inputs, not from free symbols

```
1 pair  $\leftarrow Match_r(\text{ownership}, \xi)$ ; // one pairing result feeds T2 and the no-response rule
2 if pair is ambiguous then return TimingResult(verdict = ERROR); no silent pick;
3 ;
4 if pair is a legitimate trigger/response pair then
5    $M \leftarrow \text{measurementInterval} \cap D_r$ ;
6   if  $M = \emptyset$  then return TimingResult(verdict = ERROR); a contradictory measurement model is ERROR, never PASS;
7   ;
8   if  $M \subseteq N_r$  then
9     return TimingResult(verdict = PASS, evidence = pair,  $M$ );
10  else if  $M \cap N_r = \emptyset$  then
11    return TimingResult(verdict = FAIL, evidence = pair,  $M$ );
12  return TimingResult(verdict = INCONCLUSIVE, evidence = pair,  $M$ ); open/closed endpoints preserved;
// a missing response forms a no-response only after a still-active instance reaches its deadline; this round
   not seeing it is not a timeout
13 if the obligation is still active and expired then
14   // use the earliest admissible elapsed horizon (a scalar), with the requirement's boundary convention
15   if  $N_r$  is closed at  $U_r$  then
16     expired  $\leftarrow \text{lowerHorizon} > U_r$ ;
17   else
18     expired  $\leftarrow \text{lowerHorizon} \geq U_r$ ;
19   if expired then return TimingResult(verdict = FAIL);
20   ;
21 return TimingResult(verdict = ERROR); a missing pair is not a point observation;
```

Algorithm 4: CommitCompatibleUpdate: history-compatible update and summary commit (ALG-CLTAV-07, S9)

Require: SelectSnapshot ξ ; normalized Outcome z from the resolving module; Γ ; η ; H ; precondition enforced by the caller
 ALG-CLTAV-01: the S9 guard holds (a valid observation class or a confirmed successor summary)
Ensure: successor η' , H' and the committed Γ' .currentSummary; or a named spec error

```

1  $\Gamma' \leftarrow \Gamma$ ; //  $\Gamma'$  is a copy of  $\Gamma$ ; no field is left undefined
2 ;
3 if  $z.I_z$  is a valid observation class then
4    $(\eta_c, H_c) \leftarrow$ 
      IF-HIST-UPDATE( $\eta, H, \xi.actionId, \xi.qUsedAtSelect, \xi.classesUsedAtSelect, \xi.historyVersion, z.I_z, z.postSummary$ );
      // IF-HIST-UPDATE is the sole history writer; it must agree with the intersection and the snapshot version
5   if  $H_c \neq H \cap z.I_z$  then return SPEC-ERROR; version or intersection mismatch; never overwrite  $H$  with the compatible
      set;
6   ;
7    $\eta' \leftarrow \eta_c$ ;  $H' \leftarrow H_c$ ;
8 else
9    $(\eta', H') \leftarrow$ 
      IF-HIST-UPDATE( $\eta, H, \xi.actionId, \xi.qUsedAtSelect, \xi.classesUsedAtSelect, \xi.historyVersion, NONE, z.postSummary$ );
       $H' \leftarrow H$ ;
      // a confirmed successor summary with no valid observation advances history but preserves  $H$ 
10 if  $\Gamma'.qStatus$  is KNOWN and  $z.summaryConfirmed$  is true and  $z.postSummary$  is confirmed then
11    $\Gamma'.currentSummary \leftarrow z.postSummary$ ; // this is the sole commit of the confirmed successor summary
12 return  $\eta', H', \Gamma'$ ;
```

Algorithm 5: PredictCurrent: current-observation prediction wrapper (ALG-CLTAV-02, S2)

Require: SessionContext Γ ; HistoryHandle η ; current H ; uncertainty reference U ; action library L
Ensure: PredictionResult: status OK with nonempty classesByTest, or status GAP with a reason; the session's own η is read, not rebuilt

```

1  $raw \leftarrow$  IF-PRED-OBS( $\Gamma, \eta, H, U, L$ );
  // branch on the backend tag BEFORE reading or projecting classes; a GAP result carries no classesByTest
2 if  $raw.status = GAP$  then
3   return PredictionResult(status = GAP, reason =  $raw.reason$ ); the gap stays TEST-scoped;
4  $classes \leftarrow$  ProjectOntoCurrentH( $raw.classes, H$ );
  // a declared class set that projects to no nonempty class is the same named gap, not OK and not  $score = 0$ 
5 if  $classes$  has no nonempty member then
6   return PredictionResult(status = GAP, reason = no currently valid class);
7 return PredictionResult(status = OK, classesByTest =  $classes$ , historyVersionUsed =  $\eta.version$ , uncertaintyRef =  $U$ );
```

Algorithm 6: InterpretOutcome: execution and observation coordination (ALG-CLTAV-03, S6)

Require: ExecutionResult er ; SelectSnapshot ξ ; Γ ; η ; uncertainty reference U
Ensure: a total Outcome z : effect and kind copied explicitly; skipped confirmation is NOT-EVALUATED, not an evaluated false

```
// total constructor; the executed effect and the snapshot kind are copied, never left to an implicit global
1  $z \leftarrow \text{NewOutcome}()$ ;  $z.\text{effect} \leftarrow er.\text{effect}$ ;  $z.\text{kind} \leftarrow \xi.\text{actionKind}$ ;  $z.\text{actionId} \leftarrow \xi.\text{actionId}$ ;
  // CONFIRMED-NOT-SENT and UNKNOWN-EFFECT do not call an observation or confirmation interface that did not run
2 if  $er.\text{effect} \in \{\text{CONFIRMED-NOT-SENT}, \text{UNKNOWN-EFFECT}\}$  then
3   | return  $z$ ;
4 ( $I_z, \text{effectClass}, \text{summaryConfirmed}, \text{postSummary}, \text{ownership}, \text{measurementInterval}$ )  $\leftarrow$ 
  IF-OBS-INTERPRET( $\Gamma, er.\text{record}, \text{clocks}, \varepsilon$ );
  // an invalid measurement or an interpretation ERROR is propagated to S7, not turned into a confirmed summary
5 if ownership is invalid then
6   |  $z.\text{validity} \leftarrow \text{ERROR}$ ;  $z.\text{effect} \leftarrow \text{UNKNOWN-EFFECT}$ ; return  $z$ ;
7 timing  $\leftarrow$  InterpretTimedObservation(ownership, measurementInterval,  $\xi, N_r, D_r, \text{Match}_r$ );
8 if timing.verdict = ERROR then
9   |  $z.\text{validity} \leftarrow \text{ERROR}$ ;  $z.\text{effect} \leftarrow \text{UNKNOWN-EFFECT}$ ; return  $z$ ;
  // consume the interface outputs; a normal TEST binds summaryConfirmed and postSummary explicitly, never by default
10  $z.I_z \leftarrow I_z$ ;  $z.\text{summaryConfirmed} \leftarrow \text{summaryConfirmed}$ ;  $z.\text{postSummary} \leftarrow \text{postSummary}$ ;
11 if  $\text{effectClass} = \text{NONE}$  and not  $\text{summaryConfirmed}$  then  $z.\text{effect} \leftarrow \text{UNKNOWN-EFFECT}$ ; the successor summary could not be confirmed;
12 ;
13 if  $\xi.\text{actionKind} \in \{\text{PREP}, \text{RECOVER}\}$  then
14   |  $z.\text{prepResultEvaluated} \leftarrow \text{true}$ ;
    | ( $z.\text{targetConfirmed}, z.\text{summaryConfirmed}, z.\text{prepErr}, z.\text{declaredTarget}, z.\text{evidence}, z.\text{postSummary}$ )  $\leftarrow$ 
    | IF-PREP-RECOVER( $\Gamma, \eta, \xi.\text{actionId}, er.\text{record}$ );
    | // IF-PREP-RECOVER returns values only; it does not write  $\Gamma$  or  $\eta$ ; a Prep whose target is unmet may still
    | have a confirmed successor
15 return  $z$ ; // values only; all  $\Gamma/\eta$  writes stay in ALG-CLTAV-04 and S9
```

Algorithm 7: ResolveOutcome: ERROR, retry and confirmed recovery (ALG-CLTAV-04, S7–S8)

Require: Outcome z (effect, kind, validity, confirmation); SelectSnapshot ξ ; Γ ; η
Ensure: Resolution $\delta = (\text{control}, \Gamma', \eta', \text{outcome}, \text{reason})$: Γ' initialized from Γ

```
1  $\Gamma' \leftarrow \Gamma$ ;  $\eta' \leftarrow \eta$ ; // copy/persist semantics: the returned contexts are total, never undefined
2 ;
3 if  $z.\text{validity} = \text{ERROR}$  or  $z.\text{effect} \in \{\text{CONFIRMED-NOT-SENT}, \text{UNKNOWN-EFFECT}\}$  or  $z.\text{prepErr}$  or ( $z.\text{kind}$  is Prep and
   $z.\text{prepResultEvaluated}$  and  $z.\text{summaryConfirmed}$  is false) or ( $z.\text{kind}$  is Recover and  $z.\text{prepResultEvaluated}$  and
   $z.\text{targetConfirmed}$  is false) then
  // S7 classify by kind and evaluated confirmation; do not fold CONFIRMED-NOT-SENT into UNKNOWN
4   do not exclude  $H$ ;
5   if  $z.\text{validity} = \text{ERROR}$  or  $z.\text{prepErr}$  or  $z.\text{effect} = \text{UNKNOWN-EFFECT}$  or ( $z.\text{kind}$  is Prep and  $z.\text{prepResultEvaluated}$  and
     $z.\text{summaryConfirmed}$  is false) then
6     | mark  $\eta'$  conservative-unknown;  $\Gamma'.\text{qStatus} \leftarrow \text{UNKNOWN}$ ; clear  $\Gamma'.\text{currentSummary}$ ;
7   else if  $z.\text{effect} = \text{CONFIRMED-NOT-SENT}$  then
8     | leave  $\Gamma'.\text{qStatus}$  and compatible history unchanged;
9   else
10    |  $\Gamma'.\text{qStatus}$  stays UNKNOWN; clear  $\Gamma'.\text{currentSummary}$ ; mark  $\eta'$  conservative-unknown;
    | // unconfirmed Recover target
11   $\Gamma'.\text{retryCount} \leftarrow \Gamma.\text{retryCount} + 1$ ;
12  if  $\Gamma'.\text{retryCount} \geq \Gamma'.\text{retryCap}$  then return Resolution(control = STOP,  $\Gamma', \eta', z, \text{reason} = \text{Stop-Error}$ ); not Stop-Budget;
13  ;
14  return Resolution(control = RETRY,  $\Gamma', \eta', z$ ); // an ERROR/unknown outcome reaches retry, never commit
15 else if  $z.\text{kind}$  is Recover then
  // S8 a confirmed Recover is a necessary condition for this branch; sole writer of KNOWN; the normalized
  // summary returns to S9
16   $\Gamma'.\text{qStatus} \leftarrow \text{KNOWN}$ ;  $z.\text{postSummary} \leftarrow (z.\text{declaredTarget}, z.\text{evidence})$ ;
17 return Resolution(control = COMMIT,  $\Gamma', \eta', z$ ); // a normal TEST/Prep with a valid observation commits; the normalized
  Outcome carries the confirmed postSummary to S9
```

Appendix A. Typed records and symbols

Records are explicit. An optional field is a tagged alternative, never an uninitialized variable; **NONE** (absent) differs from a tagged *false* and from an empty set. Every producer, consumer and sole writer is fixed here.

Record	Fields (producer $\rightarrow$ consumer)
PredictionResult	status OK GAP; reason (GAP); classesByTest (OK only); historyVersionUsed; uncertaintyRef. A GAP result carries no classesByTest. Producer ALG-02 $\rightarrow$ consumer ALG-05.
Decision	kind ACTION EXIT SPEC-ERROR; actionKind TEST PREP RECOVER (ACTION only); actionId, classesUsed (ACTION only); reason (EXIT/SPEC-ERROR). Producer ALG-05 $\rightarrow$ consumer ALG-01.
SelectSnapshot ξ	actionId; actionKind (from the Decision actionKind, never the ACTION/EXIT discriminant); qUsedAtSelect; historyVersion; classesUsedAtSelect; uncertaintyRef. Frozen by ALG-01 after the final action.
ExecutionResult	attemptId; record; effect $\in \{\text{NONE, CONFIRMED-NOT-SENT, UNKNOWN-EFFECT}\}$; correlationId. Producer ALG-01 $\rightarrow$ consumer ALG-03.
Outcome z	validity; effect (copied from ExecutionResult); kind (copied from ξ .actionKind); actionId; I_z (valid) or NONE; summaryConfirmed; postSummary or NONE; prepResultEvaluated; targetConfirmed; prepErr or NONE; declaredTarget or NONE; evidence or NONE; timing. Total. Producer ALG-03 $\rightarrow$ consumer ALG-04/ALG-07.
Resolution δ	control COMMIT RETRY STOP; Γ' ; η' ; outcome (normalized); reason (STOP). Γ' is initialized from Γ . Producer ALG-04 $\rightarrow$ consumer ALG-01.

Distinct concepts that must not be conflated: an unevaluated confirmation (NOT-EVALUATED) is not an evaluated *false*; no observation is not a legitimate-but-empty compatibility class; **ERROR** is not an IUT **FAIL**; an unknown summary is not the old summary. H and $\eta.H$ are one logical candidate object; **qUsedAtSelect** is not the post-effect **currentSummary**. The action *kind* (TEST/PREP/RECOVER) is separate from the action *id*; recovery is never recognized by requiring a particular id string.

Appendix B. Coordination wrappers (S2, S6, S7–S8)

PredictCurrent (ALG-CLTAV-02, S2) branches on the backend tag before reading or projecting classes and re-tags a projection that yields no nonempty class as the same named gap, so a gap never becomes an empty class collection or a score of zero; the gap stays TEST-scoped and does not by itself block eligible Prep/Recover. InterpretOutcome (ALG-CLTAV-03, S6) copies the executed effect and the snapshot kind into a total Outcome, calls IF-OBS-INTERPRET and the timed kernel ALG-CLTAV-06, and propagates an interpretation **ERROR** to S7 instead of a confirmed summary; it writes nothing. ResolveOutcome (ALG-CLTAV-04, S7–S8) returns a total Resolution initialized from Γ : an **ERROR**/unknown or unconfirmed outcome returns **RETRY**, a confirmed Recover returns **COMMIT** and is the sole writer of **KNOWN**, a normal TEST/Prep with a valid observation also commits, and the retry cap returns **STOP** (Stop-Error, not Stop-Budget). S9 is the sole commit of the confirmed successor summary, under the guard enforced by ALG-CLTAV-01. No module path starts the next prediction after a return; only S9 does.

Appendix C. Responsibility map and decision tables

Procedure	S duty	Interface / rule	Write responsibility / exit
Initialize	S0	sets Γ, η, H , resource mode, retry, U	writes Γ, η, H
EntryStop / PostUp-dateStop	S1, S10	IF-RESOURCE-STOP, IF-EQUIV	read-only; exclusive A2–A5 / P1–P5
PredictCurrent	S2	IF-PRED-OBS	read-only; tagged OK/GAP
SelectAndAdmit	S3	IF-SELECT-ADMIT	read-only; ACTION / EXIT / SPEC-ERROR
SelectSnapshot	S3	freezes ξ	writes the iteration snapshot only
ChargeOnce	S4	one charge under the mode	billed count only
ExecuteAndRecord	S5	IF-EXECUTE-RECORD	writes the execution record
InterpretOutcome	S6	IF-OBS-INTERPRET, IF-PREP-RECOVER, ALG-06	returns values only
ResolveOutcome	S7–S8	classification, retry, KNOWN	writes Γ', η' ; S8 KNOWN
CommitCompatibleUpdate	S9	IF-HIST-UPDATE	writes η', H' , commits summary

Admit	Meaning
A1	S nonempty: execute
A2	KNOWN: reducing action unaffordable: Stop-Budget
A3	KNOWN: none reducing, no Prep: Stop-NoDistinguisher
A4	UNKNOWN: Recover not eligible: Stop-Error
A5	UNKNOWN: Recover eligible, unaffordable: Stop-Budget

Stop	Priority
P1	Stop-Empty
P2	Stop-Singleton
P3	Stop-645
P4	Stop-Equivalent
P5	Stop-Budget

Appendix D. Unresolved computation kernels and successor gates

Control logic, call order, state reads/writes, budget, return values and error branches are fully specified. The internal solution techniques remain open and are abstracted by named contracts; no probability default, HMM/DTMC/Bayesian kernel or continuous-time discrete grid is introduced.

Module	Abstract kernel	Input / output / failure	Successor gate
PredictCurrent	finite conservative observation abstract and partition	classes or tagged GAP	estimator/partition algorithm
IF-HIST-UPDATE	whole-history compatible-state representation and solver	snapshot + $I_z \rightarrow H_c, \eta_c$; version/intersection mismatch is a spec error	state/clock representation and solver
InterpretTimedObservation	event collection, clock error budget, pairing	ownership + interval $\rightarrow$ TimingResult; empty domain is ERROR	collectors and timestamp handling
IF-OBS-INTERPRET	correlation, ownership, interval validity, compatible I_z	record $\rightarrow I_z$, effect, summary, ownership, interval	collectors and interval solver
IF-PREP-RECOVER	device recovery and confirmation	returns values only	protocol/device recovery steps
IF-EQUIV	provable equivalence decision	established or unknown	optional proof algorithm; not required this round
ExecuteAndRecord	protocol execution adapter, timeout, I/O	admitted action $\rightarrow$ ExecutionResult	protocol adapter and I/O

Successor order: development-ready requirements and interface acceptance; then core algorithm selection and implementation; then component verification; then configuration and experiment gates; then confirmatory experiment. None of these backends is implemented here. Specification logic is reviewable from this package; a stub walk-through is reproducible from the bounded session-loop spec; the real tool and confirmatory experiment are not implemented. Wrapper code and finite witnesses are not performance evidence for the algorithm.

Chinese step groups (same process, not a second program)

S0 初始化: Γ 为唯一状态源; q 是 $\Gamma.currentSummary$ 的别名, $qStatus$ 只记录该摘要是否已知; $retry / R_{max}$ 分别是 $retryCount / retryCap$ 的别名; 初始化 η 、 H_0 、资源模式与不确定性引用。S1 / S10 停止门: EntryStop 与 PostUpdateStop 映射既有 A2–A5 / P1–P5 决定表, 不新建停止政策。S2: ALG-CLTAV-02 先按返回标签分支再投影; PredictionGapError 只关掉 TEST 评分, 不阻止合格 Prep / Recover。S3: ALG-CLTAV-05 给出选择与准入规则; 空 TEST 集不得 $\arg \min$; 空类不阻止合格 Recover; 选定后冻结不可变快照 ξ 。S4 一次计费: 重试上限不是预算耗尽。S5: 只执行快照动作一次。S6: ALG-CLTAV-03 协调, ALG-CLTAV-06 完成时序解释; M 、判定与候选集合 I_z 是三种不同类型, 不得 $H \cap$ 判定。S7–S8: ALG-CLTAV-04 返回完整 Resolution; ERROR / 未知走 RETRY, 只有确认 Recover 走 COMMIT; S8 唯一写 KNOWN。S9: ALG-CLTAV-07 在既有守卫下做整段历史相容更新并提交已确认摘要; 不得旁路 S9 开始下一轮预测。Stop-645 表示仍未在能力/实现层关闭的具名 645 依赖义务, 不是“645 文件缺失”。